

Practical: Boston House Price Prediction using Deep Neural Network (Linear Regression)

Laboratory Practice V – Computer Engineering

1 Aim

Implement Boston housing price prediction problem by Linear Regression using Deep Neural Network. Use Boston House price prediction dataset.

1.1 Objective

Students should be able to perform Linear Regression using a Deep Neural Network on the Boston House Dataset.

2 Theory

2.1 What is Linear Regression?

Linear Regression is a statistical method to model the relationship between a **dependent variable** (output) and one or more **independent variables** (inputs). It assumes a linear relationship:

$$y = w_1x_1 + w_2x_2 + \dots + w_nx_n + b$$

Where:

- y = predicted value (house price)
- $x_1, x_2, \dots, x_n$ = input features (rooms, crime rate, etc.)
- $w_1, w_2, \dots, w_n$ = weights (learned by the model)
- b = bias

Goal: Find the best weights and bias so that predicted prices are as close as possible to actual prices.

2.2 Example of Linear Regression

Predicting house price based on features:

Input Features:

- Number of rooms (RM) = 6.5
- Crime rate (CRIM) = 0.006
- Distance to employment (DIS) = 4.09
- ... (13 features total)

Linear Regression Model:

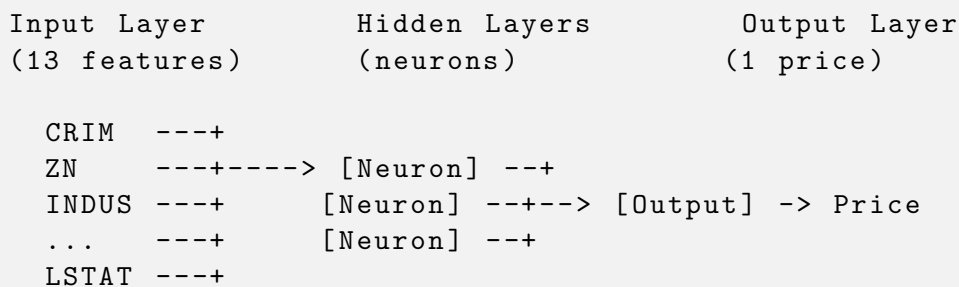
$$\text{Price} = w_1 \cdot \text{RM} + w_2 \cdot \text{CRIM} + w_3 \cdot \text{DIS} + \dots + \text{bias}$$

Predicted Price: \$24,000

2.3 What is a Deep Neural Network (DNN)?

A Deep Neural Network is a machine learning model inspired by the human brain. It has multiple layers of connected **neurons** that learn patterns from data.

2.3.1 Structure



Note: In this practical, we use a **single Dense layer with 1 neuron and linear activation** — this makes the DNN behave exactly like Linear Regression!

2.4 How DNN Works for Boston House Price Prediction

1. **Data Preprocessing:** Normalize features (mean=0, std=1) so model learns efficiently.
2. **Model Architecture:** Define network with input layer (13 features) and output layer (1 neuron).
3. **Training:** Adjust weights using **Adam optimizer** to minimize **Mean Squared Error (MSE)**.
4. **Evaluation:** Test model on unseen data and measure MSE and MAE.
5. **Prediction:** Use trained model to predict house prices.

2.5 Boston House Price Dataset

The dataset contains information about **506 houses** in Boston. It has **13 input features** and **1 output** (MEDV — median house value in \$1000s).

Feature	Description
CRIM	Per capita crime rate by town
ZN	Proportion of residential land zoned for lots > 25,000 sq.ft
INDUS	Proportion of non-retail business acres per town
CHAS	Charles River dummy variable (1 if bounds river; 0 otherwise)
NOX	Nitric oxides concentration
RM	Average number of rooms per dwelling
AGE	Proportion of owner-occupied units built before 1940
DIS	Weighted distances to five Boston employment centers
RAD	Index of accessibility to radial highways
TAX	Full-value property-tax rate per \$10,000
PTRATIO	Pupil-teacher ratio by town
B	$1000(Bk - 0.63)^2$ where Bk is proportion of black people
LSTAT	% lower status of the population
MEDV	Median value of homes in \$1000s (TARGET)

2.6 Key Concepts Used

Concept	Meaning
StandardScaler	Normalizes data: mean=0, std=1. Helps model learn faster
train_test_split	Splits data into 80% training and 20% testing
Sequential Model	Layers stacked one after another
Dense Layer	Fully connected layer (every input connects to every neuron)
Linear Activation	$f(x) = x$ (no transformation — like linear regression)
Adam Optimizer	Smart algorithm to adjust weights during training
MSE (Loss)	Mean Squared Error: average of $(actual - predicted)^2$
MAE (Metric)	Mean Absolute Error: average of $ actual - predicted $
Epochs	Number of times model sees entire training data

3 Code

Listing 1: Boston House Price Prediction using DNN

```

1 # Step 1: Import Libraries
2 import numpy as np
3 import pandas as pd
4 import matplotlib.pyplot as plt
5

```

```
6 from sklearn.model_selection import train_test_split
7 from sklearn.preprocessing import StandardScaler
8
9 import tensorflow as tf
10 from tensorflow.keras.models import Sequential
11 from tensorflow.keras.layers import Dense
12
13 # Step 2: Load Dataset
14 df = pd.read_csv("1_boston_housing.csv")
15 df.columns = df.columns.str.replace('"', '')
16 df.head()
17
18 # Step 3: Split Features and Target
19 X = df.drop("MEDV", axis=1) # 13 input features
20 y = df["MEDV"] # Target (house price)
21 print(X.shape, y.shape) # (506, 13) (506,)
22
23 # Step 4: Train-Test Split (80% train, 20% test)
24 X_train, X_test, y_train, y_test = train_test_split(
25     X, y, test_size=0.2, random_state=42
26 )
27
28 # Step 5: Feature Scaling (Standardization)
29 scaler = StandardScaler()
30 X_train = scaler.fit_transform(X_train)
31 X_test = scaler.transform(X_test)
32
33 # Step 6: Build DNN Model (Linear Regression)
34 model = Sequential()
35 model.add(Dense(1, input_shape=(X_train.shape[1],),
36     activation='linear'))
37
38 model.compile(
39     optimizer='adam',
40     loss='mse',
41     metrics=['mae']
42 )
43 model.summary()
44
45 # Step 7: Train Model (100 epochs)
46 history = model.fit(
47     X_train, y_train,
48     epochs=100,
49     batch_size=16,
50     validation_split=0.2,
51     verbose=1
52 )
53
54 # Step 8: Evaluate Model
55 loss, mae = model.evaluate(X_test, y_test)
56 print("Test Loss (MSE):", loss)
57 print("Test MAE:", mae)
58
```

```
59 # Step 9: Predictions
60 y_pred = model.predict(X_test)
61 for i in range(5):
62     print(f"Actual: {y_test.iloc[i]:.2f} | Predicted: {y_pred[i][0]:.2f}")
63
64 # Step 10: Plot Training Graphs
65 plt.plot(history.history['loss'], label='Training Loss')
66 plt.plot(history.history['val_loss'], label='Validation Loss')
67 plt.legend()
68 plt.title("Loss Graph")
69 plt.xlabel("Epochs")
70 plt.ylabel("Loss")
71 plt.show()
```

4 Code Explanation

4.1 Step 1: Import Libraries

- `numpy`, `pandas` — for data handling
- `matplotlib` — for plotting graphs
- `sklearn` — for train/test split and feature scaling
- `tensorflow.keras` — for building the neural network

4.2 Step 2: Load Dataset

Loads the Boston housing CSV file into a Pandas DataFrame. `df.head()` displays the first 5 rows to verify the data.

4.3 Step 3: Split Features and Target

- `X` = all columns except MEDV (13 input features)
- `y` = MEDV column (house price — what we want to predict)
- `X.shape = (506, 13)` means 506 houses, 13 features each

4.4 Step 4: Train-Test Split

Splits data into:

- **80% Training data** — model learns from this
- **20% Testing data** — model is evaluated on this (unseen data)

`random_state=42` ensures the split is the same every time we run the code.

4.5 Step 5: Feature Scaling (Standardization)

- `StandardScaler` transforms each feature to have **mean=0** and **std=1**
- `fit_transform` on training data: calculates mean/std and transforms
- `transform` on test data: uses the **same** mean/std from training (important!)

Why scale? Features have very different ranges (e.g., `CRIM=0.006`, `TAX=296`). Without scaling, the model struggles to learn.

4.6 Step 6: Build DNN Model

- `Sequential()` — creates a model where layers are stacked linearly
- `Dense(1, activation='linear')` — single neuron with linear activation
- This single neuron computes: $\text{Price} = w_1x_1 + w_2x_2 + \dots + w_{13}x_{13} + b$
- Total parameters: 13 weights + 1 bias = 14
- `optimizer='adam'` — Adam optimizer to update weights
- `loss='mse'` — minimize Mean Squared Error
- `metrics=['mae']` — track Mean Absolute Error

4.7 Step 7: Train Model

- `epochs=100` — model sees training data 100 times
- `batch_size=16` — processes 16 samples at a time
- `validation_split=0.2` — uses 20% of training data for validation
- During training, loss decreases from ~ 600 to ~ 450 (model is learning!)

4.8 Step 8: Evaluate Model

Tests the model on **unseen** test data and prints:

- **Test MSE:** ~ 448.8 (average squared error)
- **Test MAE:** ~ 20.33 (average absolute error in \$1000s)

4.9 Step 9: Predictions

Compares actual vs predicted prices for first 5 test samples.

4.10 Step 10: Plot Loss Graph

Plots training loss and validation loss over 100 epochs. Both should **decrease** — indicating the model is learning.

5 Output

5.1 Model Summary

```
Model: "sequential"
+-----+-----+-----+
| Layer (type) | Output Shape | Param # |
+-----+-----+-----+
| dense (Dense) | (None, 1) | 14 |
+-----+-----+-----+
Total params: 14
Trainable params: 14
Non-trainable params: 0
```

5.2 Training Output (Selected Epochs)

```
Epoch 1/100 - loss: 621.15 - mae: 22.96 - val_loss: 559.12
Epoch 25/100 - loss: 571.50 - mae: 22.44 - val_loss: 524.35
Epoch 50/100 - loss: 534.20 - mae: 21.90 - val_loss: 497.50
Epoch 75/100 - loss: 503.84 - mae: 21.38 - val_loss: 474.12
Epoch 100/100 - loss: 477.24 - mae: 20.86 - val_loss: 452.14
```

5.3 Evaluation

```
Test Loss (MSE): 448.8063
Test MAE: 20.3290
```

5.4 Predictions

```
Actual: 23.60 | Predicted: 4.87
Actual: 32.40 | Predicted: 8.55
Actual: 13.60 | Predicted: -2.48
Actual: 22.80 | Predicted: 5.95
Actual: 16.10 | Predicted: -3.80
```

Note: The simple single-neuron model (pure linear regression) has limited accuracy. Adding more hidden layers would improve predictions significantly.

6 How to Run

This code runs in **Google Colab** or **Jupyter Notebook** with TensorFlow installed.

6.1 On Google Colab

1. Go to colab.research.google.com
2. Upload the .ipynb notebook file
3. Upload the `1.boston_housing.csv` dataset to Colab
4. Run all cells (Runtime → Run all)

6.2 On Local Machine (Anaconda/Jupyter)

```
# Install required libraries
pip install numpy pandas matplotlib scikit-learn tensorflow

# Open Jupyter Notebook
jupyter notebook

# Open the .ipynb file and run all cells
```

6.3 On Fresh Ubuntu PC (Step by Step)

If you have a brand new Ubuntu system with nothing installed, follow these steps:

1. Update system packages:

```
sudo apt update
sudo apt upgrade -y
```

2. Install Python 3 and pip:

```
sudo apt install python3 python3-pip python3-venv -y
```

3. Verify Python installation:

```
python3 --version
pip3 --version
```

4. Create a virtual environment (recommended):

```
python3 -m venv dl_env
source dl_env/bin/activate
```

5. Install all required libraries:

```
pip install numpy pandas matplotlib scikit-learn tensorflow
jupyter
```

Note: This may take several minutes as TensorFlow is a large library (~500 MB).

6. Create a project folder and navigate to it:

```
mkdir ~/DL_Practical
cd ~/DL_Practical
```

7. Copy the dataset and notebook file:

```
# Copy CSV and .ipynb to this folder
cp /path/to/1_boston_housing.csv .
cp /path/to/1_Boston_House_Price_Prediction.ipynb .
```

8. Launch Jupyter Notebook:

```
jupyter notebook
```

This will open a browser window with Jupyter interface.

9. **Open the notebook:**

Click on `1_Boston_House_Price_Prediction.ipynb` in the Jupyter browser.

10. **Run all cells:**

Click **Cell** → **Run All** or press **Shift+Enter** on each cell one by one.

11. **Alternative — Run as Python script:**

If you don't want to use Jupyter, save the code as a `.py` file and run:

```
python3 boston_predict.py
```

Troubleshooting:

- If `pip install tensorflow` fails, try: `pip install tensorflow-cpu`
- If Jupyter doesn't open browser, copy the URL from terminal and paste in browser
- Make sure the CSV file (`1.boston_housing.csv`) is in the **same folder** as the notebook

7 Simple Explanation (Hinglish)

7.1 Is Practical Ka Aim Kya Hai?

Humein Boston city ke **506 gharon ki price predict** karni hai. Har ghar ke baare mein 13 cheezein pata hain (jaise kitne rooms hain, crime rate kitna hai, tax kitna hai, school ka ratio kya hai, etc.). In 13 features ko dekhke model ko seekhna hai ki ghar ki price kya hogi.

Yeh ek **Regression problem** hai — matlab output ek number hoga (price), category nahi.

7.2 Karna Kya Hai? (Step by Step)

7.2.1 Cell 1: Import Libraries

Sabse pehle zaruri libraries import karni hain:

- `numpy` — maths operations ke liye (arrays, calculations)
- `pandas` — CSV file padhne aur data handle karne ke liye
- `matplotlib` — graph/chart banana ke liye
- `sklearn` — data split karne aur scaling ke liye
- `tensorflow/keras` — neural network banane aur train karne ke liye

Simple: Yeh sab tools hain jo humein kaam mein lagenge.

7.2.2 Cell 2: Load Dataset

```
df = pd.read_csv("1_boston_housing.csv")
df.head()
```

CSV file se data load hota hai ek table (DataFrame) mein. `df.head()` se pehle 5 rows dikhti hain — taaki verify kar sakein ki data sahi load hua hai ya nahi.

Simple: Excel jaisi file ko Python mein load kiya.

7.2.3 Cell 3: Split Features and Target

```
X = df.drop("MEDV", axis=1)    # 13 features
y = df["MEDV"]                # house price
```

- X = wo sab columns jo model ko **input** mein milenge (13 features)
- y = wo column jise model ko **predict** karna hai (MEDV = house price)
- X.shape = (506, 13) matlab 506 gharon ka data hai, har ghar ke 13 features

Simple: Input aur Output alag kiye — “yeh diya, yeh nikalo” wala system.

7.2.4 Cell 4: Train-Test Split

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)
```

Data ko 2 parts mein toda:

- **80% Training** — model isse seekhega (404 houses)
- **20% Testing** — model ki ability test karenge (102 houses)

Simple: Jaise exam ke liye practice questions (train) aur actual exam (test) hota hai.

7.2.5 Cell 5: Feature Scaling (StandardScaler)

```
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

Har feature ki value ko normalize kiya: **mean=0, std=1**.

Kyun? Ek feature ki range 0-1 hai (CRIM=0.006) aur doosre ki 100-700 (TAX=296). Agar aise hi denge toh model confuse hoga — bade number ko zyada importance dega. Scaling se sab features **equal footing** pe aa jaate hain.

Simple: Sabko ek hi scale pe laaya taaki koi feature dominate na kare.

7.2.6 Cell 6: Build DNN Model

```
model = Sequential()
model.add(Dense(1, input_shape=(13,), activation='linear'))
model.compile(optimizer='adam', loss='mse', metrics=['mae'])
```

- Sequential() = layers ek ke baad ek lagao
- Dense(1) = sirf **1 neuron** hai — yeh **linear regression** hai!
- Is neuron ka formula: $Price = w_1 \times CRIM + w_2 \times ZN + \dots + w_{13} \times LSTAT + bias$
- Total 14 parameters hain (13 weights + 1 bias) — yahi model seekhega
- adam = smart optimizer jo weights adjust karega
- mse = Mean Squared Error — loss function (kitna galat hai)
- mae = Mean Absolute Error — metric (average galti kitni hai)

Simple: Ek equation banai jo 13 features ka weighted sum karke price nikalegi.

7.2.7 Cell 7: Train Model

```
history = model.fit(X_train, y_train, epochs=100,
                    batch_size=16, validation_split=0.2)
```

- `epochs=100` — model poora training data 100 baar dekhega
- `batch_size=16` — ek baar mein 16 houses ka data process karega
- `validation_split=0.2` — training data ka 20% validation ke liye alag rakha
- Har epoch mein model weights adjust karta hai taaki loss kam ho
- Loss: 621 (epoch 1) → 477 (epoch 100) — matlab model seekh raha hai!

Simple: Jaise student baar baar practice karta hai, model bhi 100 baar data dekhke seekhta hai.

7.2.8 Cell 8: Evaluate Model

```
loss, mae = model.evaluate(X_test, y_test)
print("Test Loss (MSE):", loss)
print("Test MAE:", mae)
```

Model ko **naye, unseen data** pe test kiya:

- **Test MSE** ≈ 448 — average squared error
- **Test MAE** ≈ 20.3 — matlab average mein \$20,300 ki galti hai

Simple: Exam diya — kitna score aaya check kiya.

7.2.9 Cell 9: Predictions

```
y_pred = model.predict(X_test)
# Actual: 23.60 | Predicted: 4.87
```

Actual price vs model ki predicted price compare ki. Is simple model mein accuracy kam hai kyunki sirf 1 neuron hai (pure linear regression). Agar hidden layers add karein toh accuracy bahut badh jaayegi.

Simple: Model ne kya predict kiya vs asli price kya thi — compare kiya.

7.2.10 Cell 10: Plot Loss Graph

```
plt.plot(history.history['loss'], label='Training Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.show()
```

Training loss aur validation loss ka graph banaya. Dono lines **neeche jaani chahiye** — matlab model improve ho raha hai. Agar validation loss upar jaane lage toh **overfitting** ho rahi hai.

Simple: Graph se dekhte hain ki model time ke saath seekh raha hai ya nahi.

7.3 Final Outcome Kya Milta Hai?

1. **Trained Model** — jo 13 features dekhke ghar ki price predict kar sakta hai
2. **MSE aur MAE values** — model kitna accurate hai
3. **Loss Graph** — model kaise improve hua over 100 epochs
4. **Actual vs Predicted comparison** — real prices vs model ki predictions

Bottom Line: Is practical mein humne dekha ki ek simple neural network (1 neuron + linear activation) actually **Linear Regression** hi hai — bas TensorFlow/Keras framework use karke banaya. Yeh Deep Learning ka sabse basic foundation hai.